

The following functions define the logic:

- If file has been created with a folder.
- If file has been modified.

In both the cases, the following piece of code just compares the original and the destination's dictionaries. If any file gets created or modified, then the interpreter copies the file from source and pastes it into the destination.

```
def LogicCompare():
    from1 = All_Config.get("from")
    to= All_Config.get("to")
    Dest_dict = Destination()
    if Dest_dict:
        Source_dict = OriginalFiles()
        remaining_files = set(Source_dict.keys())- set(Dest_
dict.keys())
        remaining_files= [Source_dict.get(k) for k in
remaining_files]

        for file_path in remaining_files:
            try:
                log1.logger.info('File: {0}'.format(file_path))
                dir, file = file_path.rsplit("\\",1)
                rel_dir = from1.rsplit("\\",1)[1]
                rel_dir1 = dir.replace(from1,"")
                dest_dir = to+rel_dir+"\\")+rel_dir1
                if not os.path.isdir(dest_dir):
                    os.makedirs(dest_dir)
                CopyFiles(file_path, dest_dir)
            except Exception as e:
                log1.logger.error('Error LogicCompare: {0}'.
format(e))
```

The following piece of code uses *loop* to run the code again and again:

```
i = 0
while True:
    if i >= repeat:
        break
    LogicCompare()
    time.sleep(Freq)
    i = i +1
```

Let us see the content of file Sync1.ini

```
[Para]
From = K:\testing1
To = E:\
Freq = 1
Total_time = 5
```

In the above code:

From: Specifies what the source means and takes the backup of the *testing1* folder.

To: Specifies where to take the backup.

Freq: Takes the backup after a specified minute.

Total_time: Runs the code for *Total_time* minutes.

Let us look at the code of *logger1.py*:

```
import logging
logger = logging.getLogger("Mohit")
logger.setLevel(logging.INFO)
fh = logging.FileHandler("Sync.log")
formatter = logging.Formatter('%(asctime)s - %(levelname)s -
%(message)s')
fh.setFormatter(formatter)
logger.addHandler(fh)
```

The above code is very simple and will work in *INFO* mode.

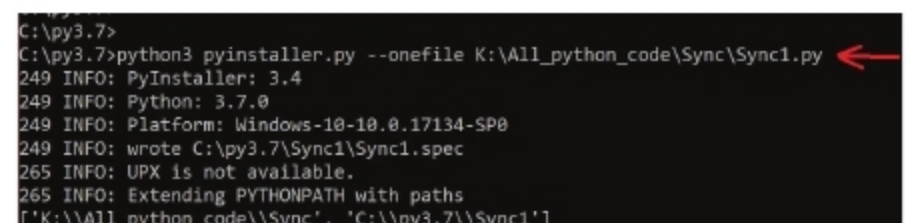
If you don't want to bother running the code using the interpreter, make a Windows exe file and this will work as a command.

In order to convert this, let us take the help of *pyinstaller*. I have already installed that module.

The command in Figure 1 converts your code into an *exe* file. To run it, the Python interpreter is not needed.

How to run the program

After executing the command as shown in Figure 1, check the folder named *Sync*. In this folder, check the folder named *dist*, where you will get the *.exe* file. Now copy this *.exe* file and



```
C:\py3.7>
C:\py3.7>python3 pyinstaller.py --onefile K:\All_python_code\Sync\Sync1.py
249 INFO: PyInstaller: 3.4
249 INFO: Python: 3.7.0
249 INFO: Platform: Windows-10-10.0.17134-SP0
249 INFO: wrote C:\py3.7\Sync1\Sync1.spec
265 INFO: UPX is not available.
265 INFO: Extending PYTHONPATH with paths
['K:\All_python_code\Sync', 'C:\py3.7\Sync1']
```

Figure 1: Making an *exe* file using *pyinstaller*

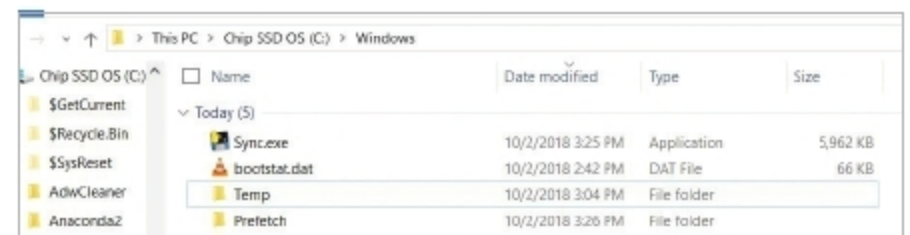


Figure 2: Place the *exe* file in the Windows folder



Figure 3: CMD default path



Figure 4: Sync command